

Software engineering

Unit testing

Skill progressions by applying material:

Unit testing +3-5



The art of unit testing

A customer walks into a bar
and asks where the
bathroom is. The bar bursts
into flames, killing everyone.

Basic ideas

A ***unit of work*** is everything that happens between the invocation of a public method and a single noticeable result through its public APIs and behaviour.

A ***unit test*** is an automated piece of code that invokes the unit of work being tested and then checks some assumptions about a single result of that unit.

An ***integration test*** is testing a unit of work without having full control over all of it and using one or more of its real dependencies, such as time, network, database, threads, random number generators, and so on.

A good unit test

- It should document what your code is supposed to be doing
- It should be automated and repeatable.
- Anyone should be able to run it at the push of a button.
- It should be easy to implement.
- It should test one thing.
- It should run quickly.
- It's trustworthy, readable, and maintainable.
- It should be consistent in its results (it always returns the same result if you don't change anything between runs).
- It should have full control of the unit under test.
- It should be fully isolated (runs independently of other tests).
- When it fails, it should be easy to detect what was expected and determine how to pinpoint the problem.

First test

```
[Test]
public void IsValidFileName_BadExtension_ReturnsFalse()
{
    //Arrange objects, creating and setting them up, as necessary.
    LogAnalyzer analyzer = new LogAnalyzer();

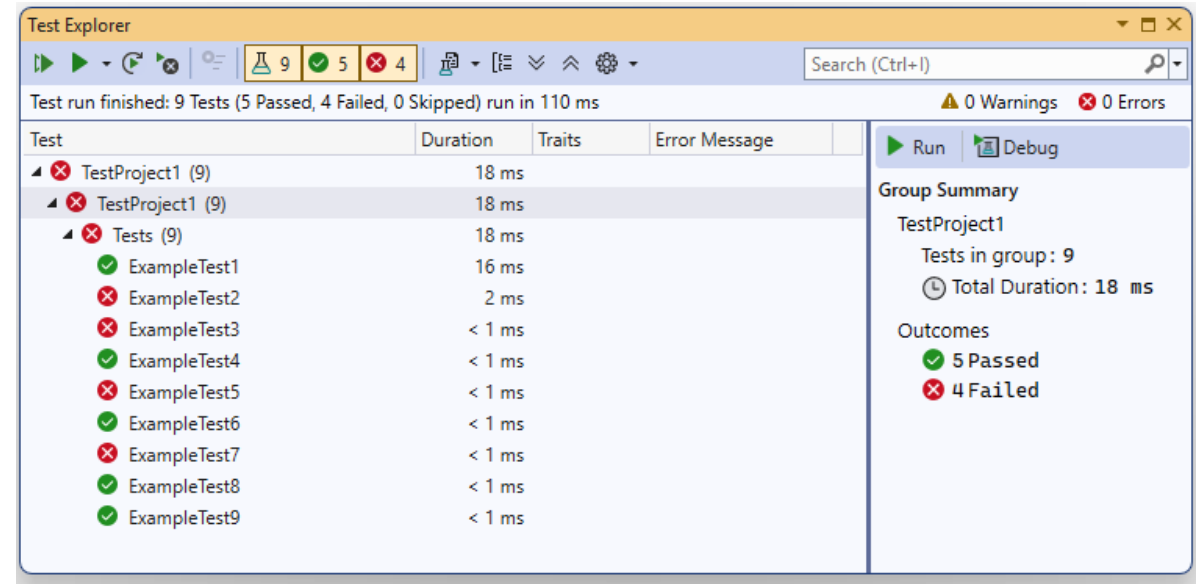
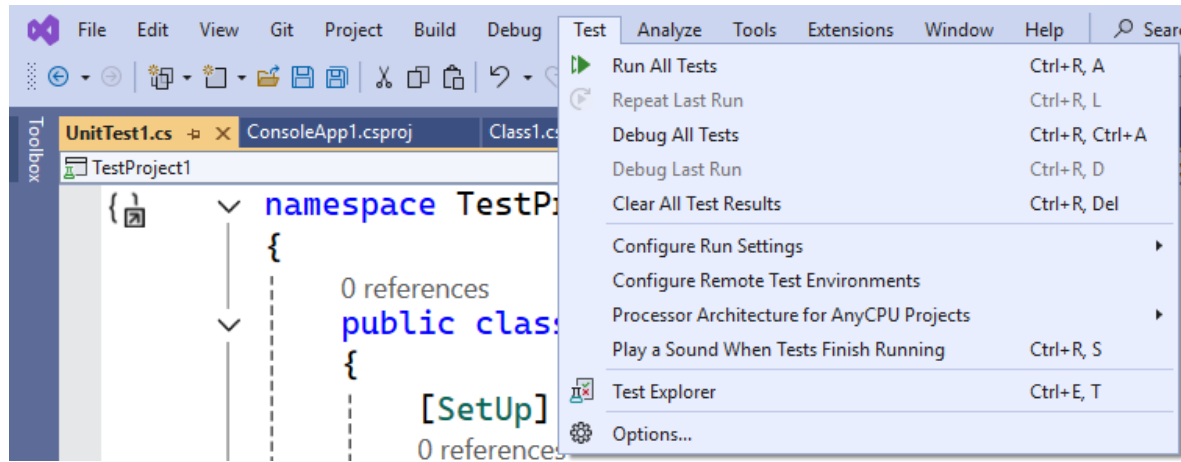
    //Act on an object.
    bool result = analyzer.IsValidLogFileName("fileWithBadExtension.foo");

    //Assert on the result.
    Assert.False(result);
}
```

What to test?

- What: Logic!
- What not:
 - Properties / Getters / setters unless you put logic into them
 - Methods without logic
 - Private Methods
 - Frameworks / external APIs, unless doing exploratory tests to understand that framework better

Anyone should be able to run it at the push of a button



It should be automated and repeatable.

All pipelines > New release pipeline

Pipeline Tasks Variables Retention Options History

Stage 1
Deployment process

Agent job
Run on agent

VsTest - testRun
Visual Studio Test

Visual Studio Test ⓘ

Version 2.*

Display name *
VsTest - testRun

Test selection ^

Select tests using * ⓘ

Test run

Test Run ⓘ

\$(test.RunId)

Iteration 1: Web Team F

Web Team Plan

Automated tests

Test suite: Automated tests (S...

Tests Charts Outcome All Tester All Confi

+ New Add existing X

Outcome	Order	ID ↑	Title
✓ Passed	1	60	IsEnabled_FeatureParaml
✗ Failed	2	61	MissingValuesInRESTJso
✗ Failed	3	62	ParamsMismatchInRESTJs
✗ Failed	4	63	LogicalSiteNameTest_Em
✓ Passed	5	65	ProcessValidJsonInSwagg
✓ Passed	6	66	LogicalSiteNameTest_Cor

Recent test results

Outcome	Configuration
✗ Failed	Windows 10
✗ Failed	Windows 10
✓ Passed	Windows 10
✓ Passed	Windows 10
✓ Passed	Windows 10

It should test one thing.



```
[Test]
public void IsValidFileName_BadExtension_ReturnsFalse()
{
    LogAnalyzer analyzer = new LogAnalyzer();

    bool result = analyzer.IsValidLogFileName("abc.def");

    Assert.Equals(false, result);
}
```

```
[Test]
public void TooManyAsserts()
{
    var calculator = new Calculator();

    Assert.Equals(8, calculator.Add(5, 3));
    Assert.Equals(6, calculator.Subtract(10, 4));
    Assert.Equals(42, calculator.Multiply(6, 7));
    Assert.Equals(4, calculator.Divide(20, 5));
    Assert.Equals(8, calculator.Power(2, 3));
    Assert.Equals(1, calculator.Modulus(10, 3));
    Assert.Equals(true, calculator.IsEven(8));
    Assert.Equals(true, calculator.IsOdd(7));
}
```

It should be easy to implement: Parameterized tests

```
[TestCase("filewithgoodextension.SLF")]  
[TestCase("filewithgoodextension.slf")]  
public void IsValidLogFileName_ValidExtensions_ReturnsTrue(string file)  
{  
    LogAnalyzer analyzer = new LogAnalyzer();  
    bool result = analyzer.IsValidLogFileName(file);  
    Assert.True(result);  
}
```

It should be easy to implement: Setup and tear down

```
[TestFixture]
public class LogAnalyzerTests
{
    private LogAnalyzer analyzer = null;

    [SetUp]
    public void Setup()
    {
        analyzer = new LogAnalyzer();
    }

    [TearDown]
    public void TearDown()
    {
        //never used in unit tests,
        //only in integration tests
    }
}
```

```
[Test]
public void IsValidFileName_validFileLowerCased_ReturnsTrue()
{
    bool result = analyzer.IsValidLogFileName("whatever.slf");
    Assert.IsTrue(result, "filename should be valid!");
}

[Test]
public void IsValidFileName_validFileUpperCased_ReturnsTrue()
{
    bool result = analyzer.IsValidLogFileName("whatever.SLF");
    Assert.IsTrue(result, "filename should be valid!");
}
```

Factory methods

```
[Test]
public void Sum_Default_ReturnsZero()
{
    Calculator calculator = MakeCalculator();
    int sum = calculator.Sum();
    Assert.AreEqual(0, sum);
}
```

```
[Test]
public void Add_WhenCalled_ChangesSum()
{
    Calculator calculator = MakeCalculator();
    calculator.Add(1);
    int sum = calculator.Sum();
    Assert.AreEqual(1, sum);
}
```

```
private static Calculator MakeCalculator()
{
    return new Calculator();
}
```

**BEWARE
OF DOG
CAT IS NOT
TRUSTWORTHY EITHER**

Logic in tests:

“Let me just copy-paste this `string.Format()`”

- Absolutely no: if / switch / for / foreach / while / Random in your tests
- Test bugs are harder to find
- Tests with random numbers can fail then pass
- Probably does multiple things
- Harder to read, understand and maintain

```
[Test]
public void Untrustworthy()
{
    var random = new Random();
    var calculator = new Calculator();

    for (int i = 0; i < 10; i++)
    {
        int a = random.Next(0, 100);
        int b = random.Next(0, 100);

        double result = calculator.Divide(a, b);

        Assert.Equals(a / b, result);
    }
}
```

It should be consistent in its results

```
private static int sharedCounter = 0;
```

```
[Test]
```

```
public void TestIncrementCounter()
```

```
{
```

```
    sharedCounter++;
```

```
    Assert.Equals(1, sharedCounter);
```

```
}
```

```
[Test]
```

```
public void TestResetCounter()
```

```
{
```

```
    sharedCounter = 0;
```

```
    Assert.Equals(0, sharedCounter);
```

```
}
```

```
[Test]
```

```
public void TestDoubleIncrement()
```

```
{
```

```
    sharedCounter++;
```

```
    sharedCounter++;
```

```
    Assert.Equals(2, sharedCounter);
```

```
}
```

Trickiness of exceptions

```
[Test]
```

```
[ExpectedException(typeof(ArgumentException), ExpectedMessage = "filename has to be  
provided")]
```

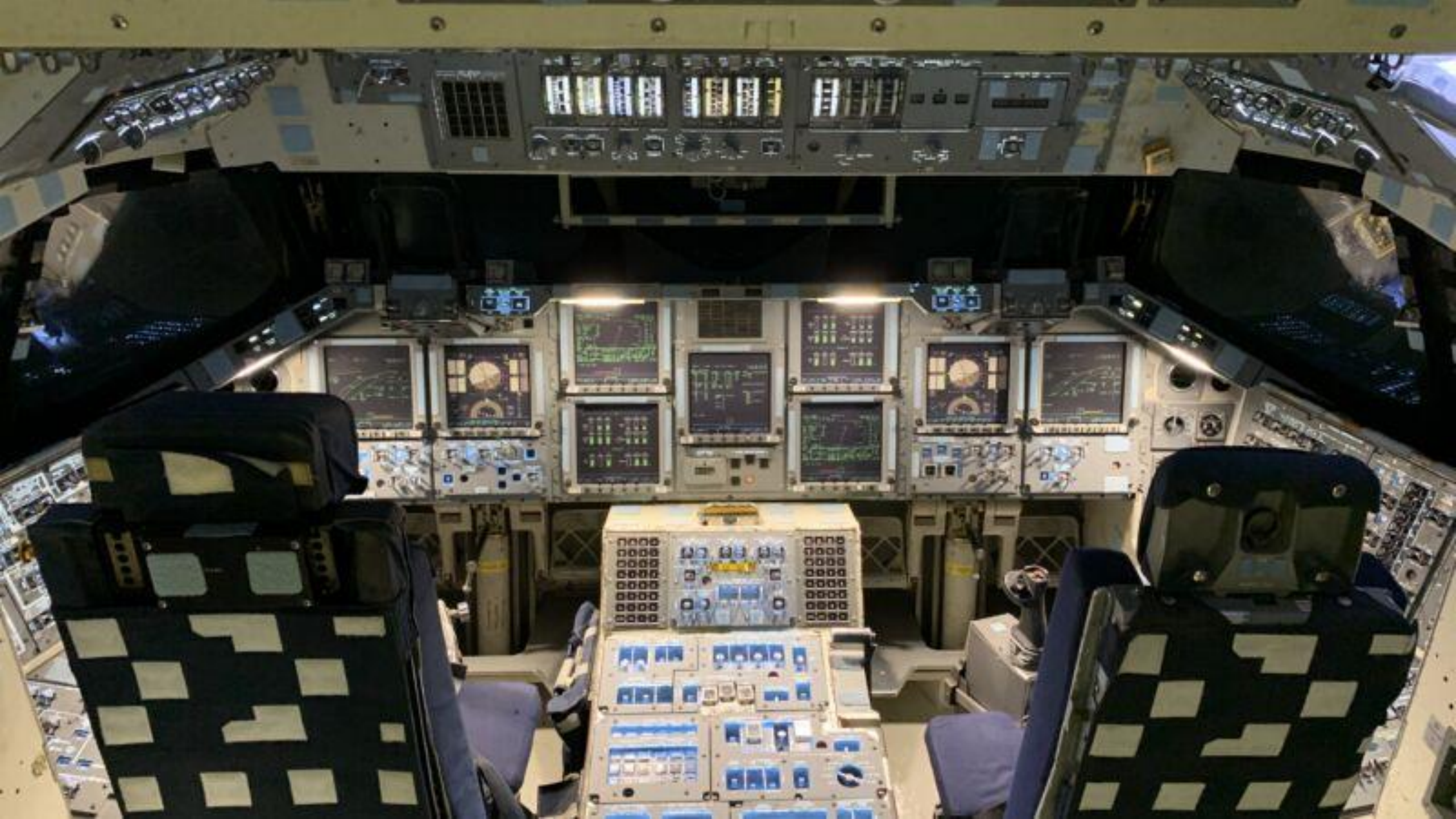
```
public void IsValidFileName_EmptyFileName_ThrowsException()  
{  
    analyzer.IsValidLogFileName(string.Empty);  
}
```

```
[Test]
```

```
public void IsValidFileName_EmptyFileName_Throws()  
{  
    var exception = Assert.Catch<Exception>(() => analyzer.IsValidLogFileName(""));  
    StringAssert.Contains("filename has to be provided", exception.Message);  
}
```

A good unit test

- It should document what your code is supposed to be doing
- It should be automated and repeatable.
- Anyone should be able to run it at the push of a button.
- When it fails, it should be easy to detect what was expected and determine how to pinpoint the problem.
- It should be easy to implement.
- It should test one thing.
- It should run quickly. It's trustworthy, readable, and maintainable.
- It should be consistent in its results (it always returns the same result if you don't change anything between runs).
- It should have full control of the unit under test.
- It should be fully isolated (runs independently of other tests).



```
private GlobalSettings globalSettings;

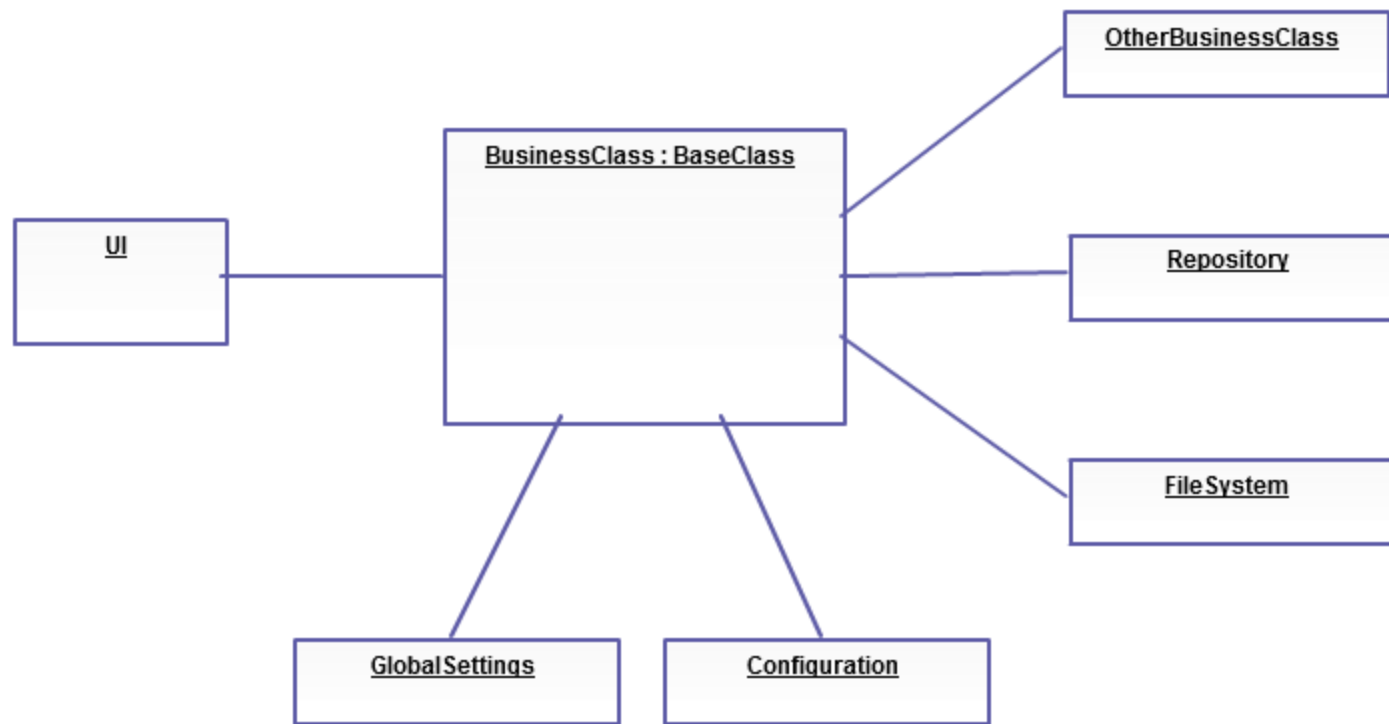
public LogAnalyzer()
{
    globalSettings = new GlobalSettings();
}

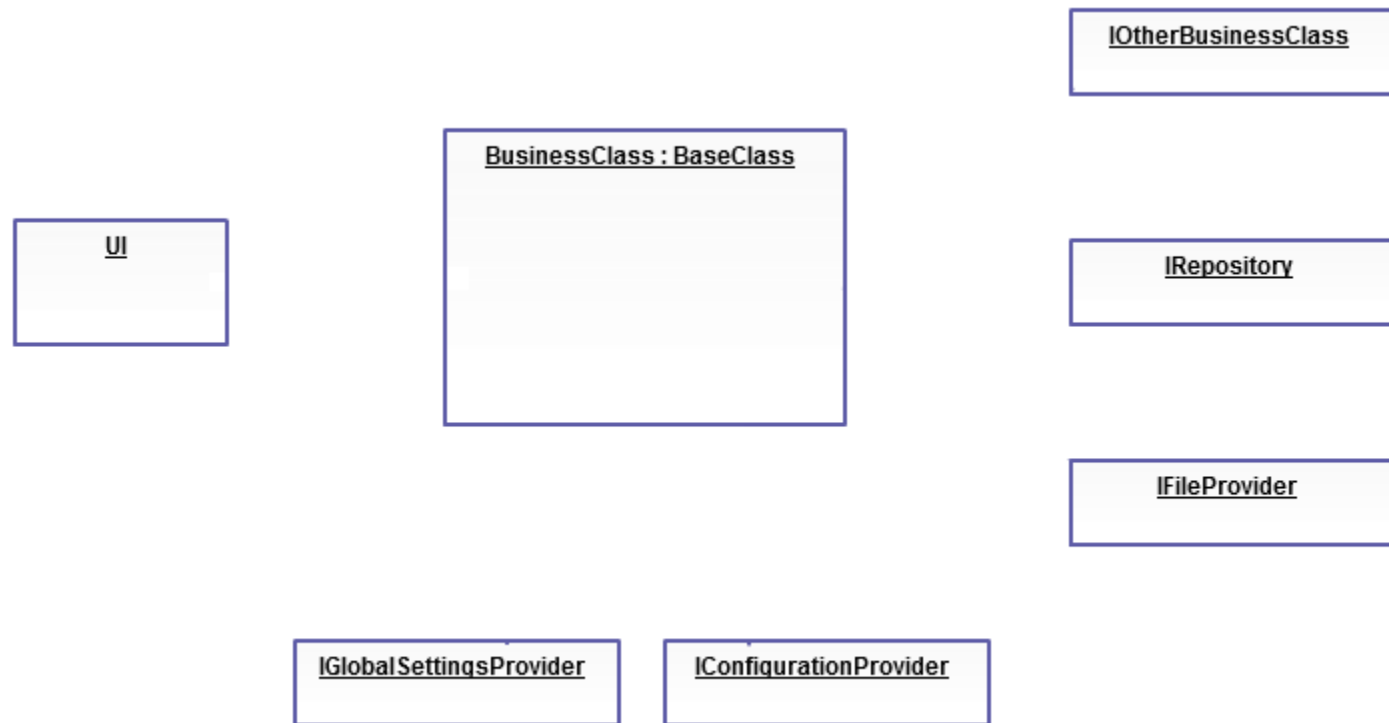
public bool LogFileExists(string fileName)
{
    return File.Exists(Settings.Path + fileName);
}

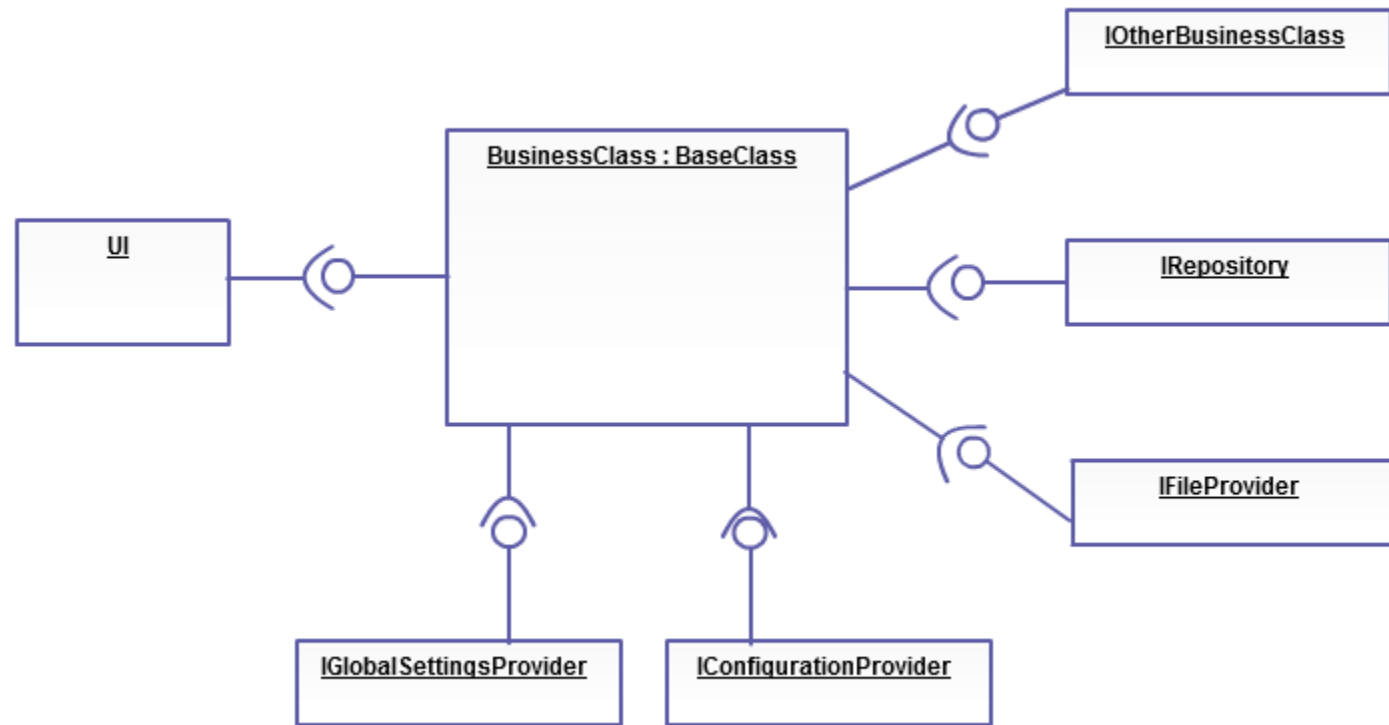
public void LogServiceError()
{
    if (!LogFileExists(globalSettings.GetLogFileName()))
    {
        string error = new SomeService().GetError();
        new Repository().Save(error);
    }
}
```

Fakes / Stubs / Mocks

“There is no object-oriented problem that cannot be solved by adding a layer of indirection, except of course for too many layers of indirection.”







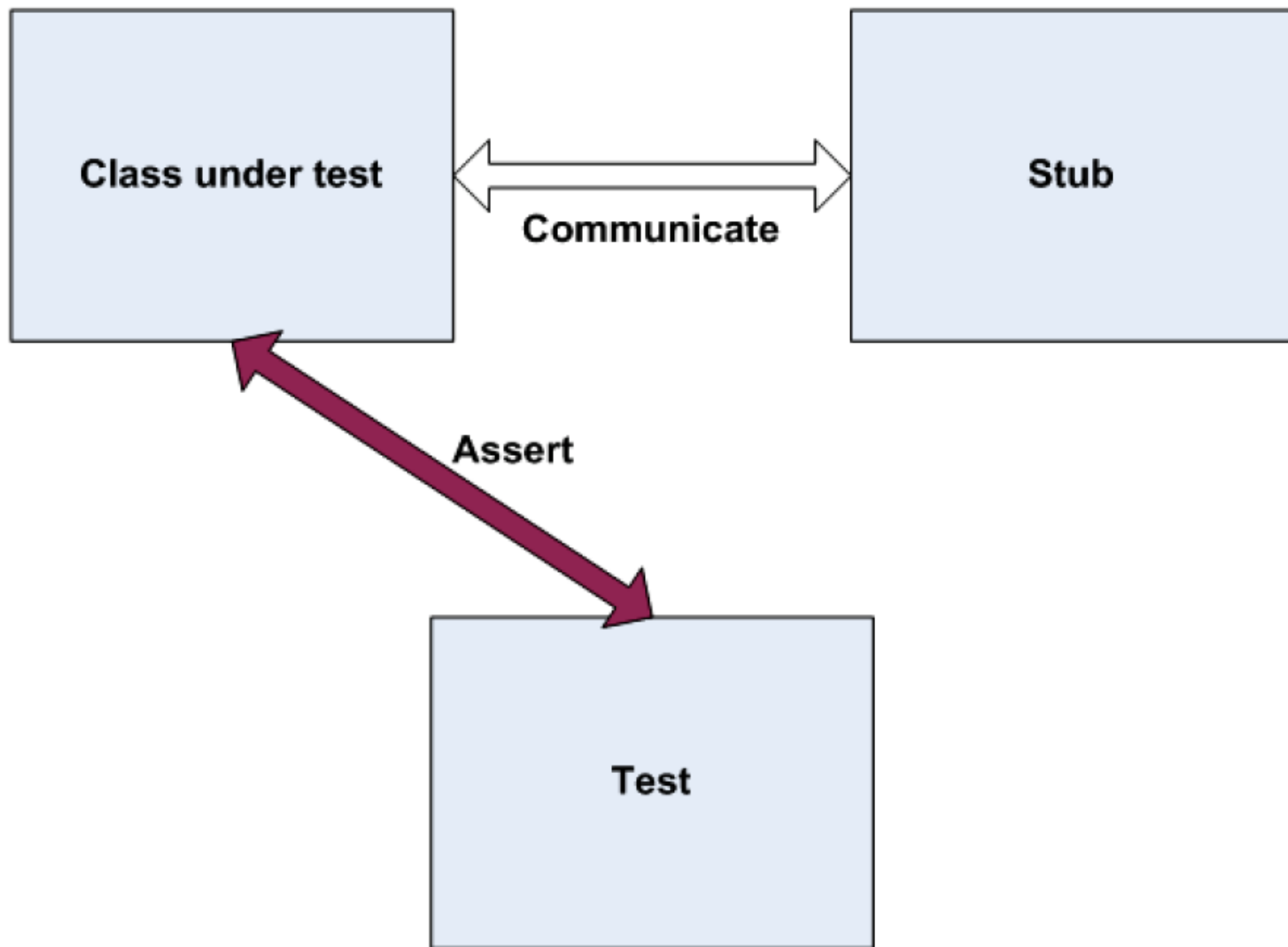
```

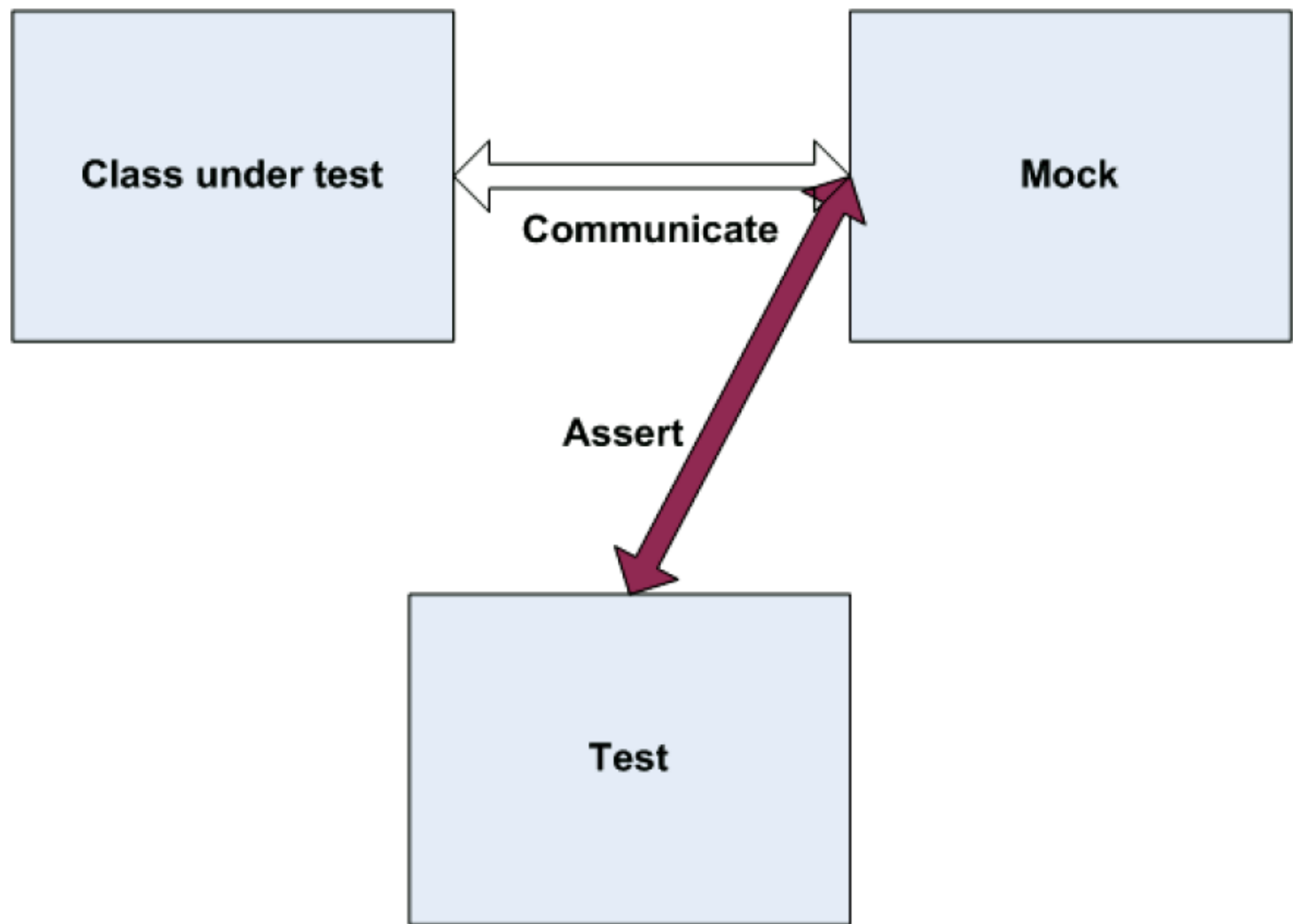
public class LogAnalyzer
{
    private IWebService service;
    private IEmailService email;

    public LogAnalyzer(IWebService service, IEmailService email)
    {
        this.email = email;
        this.service = service;
    }

    public void Analyze(string fileName)
    {
        if (fileName.Length < 8)
        {
            try
            {
                service.LogError("Filename too short:" + fileName);
            }
            catch (Exception e)
            {
                email.SendEmail(new EmailInfo
                {
                    To = "someone@somewhere.com",
                    Subject = "can't log",
                    Body = e.Message
                });
            }
        }
    }
}

```





Stubs & Mocks

```
public interface IWebService
{
    void LogError(string message);
}

public class EmailInfo
{
    public string Body;
    public string To;
    public string Subject;
}

public interface IEmailService
{
    void SendEmail(EmailInfo emailInfo);
}
```

```
public class FakeWebService : IWebService
{
    public Exception ToThrow;
    public void LogError(string message)
    {
        if (ToThrow != null)
        {
            throw ToThrow;
        }
    }
}

public class FakeEmailService : IEmailService
{
    public EmailInfo email = null;
    public void SendEmail(EmailInfo emailInfo)
    {
        email = emailInfo;
    }
}
```

```

public class LogAnalyzer
{
    private IWebService service;
    private IEmailService email;

    public LogAnalyzer(IWebService service, IEmailService email)
    {
        this.email = email;
        this.service = service;
    }

    public void Analyze(string fileName)
    {
        if (fileName.Length < 8)
        {
            try
            {
                service.LogError("Filename too short:" + fileName);
            }
            catch (Exception e)
            {
                email.SendEmail(new EmailInfo
                {
                    To = "someone@somewhere.com",
                    Subject = "can't log",
                    Body = e.Message
                });
            }
        }
    }
}

```

```

[TestFixture]
public class LogAnalyzerTests
{
    [Test]
    public void Analyze_WebServiceThrows_SendsEmail()
    {
        FakeWebService stubService = new FakeWebService();
        stubService.ToThrow = new Exception("fake exception");
        FakeEmailService mockEmail = new FakeEmailService();
        LogAnalyzer log = new LogAnalyzer(stubService, mockEmail);
        string tooShortFileName = "abc.ext";

        log.Analyze(tooShortFileName);

        EmailInfo expectedEmail = new EmailInfo
        {
            Body = "fake exception",
            To = "someone@somewhere.com",
            Subject = "can't log"
        };
        Assert.AreEqual(expectedEmail, mockEmail.email);
    }
}

```

Injection

```
public class LogAnalyzer
{
    private IFileSystemHelper fileSystemHelper;

    public LogAnalyzer(IFileSystemHelper fileSystemHelper)
    {
        this.fileSystemHelper = fileSystemHelper;
    }

    public IFileSystemHelper OptionalFileSystemHelper {get; set;}

    public virtual IFileSystemHelper GetSystemHelper()
    {
        if (fileSystemHelper != null)
        {
            return fileSystemHelper;
        }
        return new FileSystemHelper();
    }
}
```

```
[Test]
public void LogfileExists_ConstructorInjection()
{
    var fakeFileSystemHelper = new FakeFileSystemHelper();
    var logAnalyzer = new LogAnalyzer(fakeFileSystemHelper);
    Assert.IsTrue(logAnalyzer.LogFileExists("anything"));
}

[Test]
public void LogfileExists_PropertyInjection()
{
    var logAnalyzer = new LogAnalyzer();
    logAnalyzer.OptionalFileSystemHelper = new FakeFileSystemHelper();
    Assert.IsTrue(logAnalyzer.LogFileExists("anything"));
}

internal class TestableLogAnalyzer : LogAnalyzer
{
    override public IFileSystemHelper GetSystemHelper()
    {
        return new FakeFileSystemHelper();
    }
}

[Test]
public void LogfileExists_DerivedClassOverride()
{
    var logAnalyzer = new TestableLogAnalyzer();
    Assert.IsTrue(logAnalyzer.LogFileExists("anything"));
}
```

Isolation (mocking) frameworks

```
[Test]
public void Analyze_TooShortFileName_CallLogger()
{
    FakeLogger logger = new FakeLogger();
    LogAnalyzer analyzer = new LogAnalyzer(logger);
    analyzer.Analyze("a.txt");
    StringAssert.Contains("too short", logger.LastError);
}

class FakeLogger : ILogger
{
    public string LastError;
    public void Log(string message)
    {
        LastError = message;
    }
}
```

```
//Using NSubstitute
[Test]
public void Analyze_TooShortFileName_CallLoggerSubstitute()
{
    ILogger logger = Substitute.For<ILogger>();
    LogAnalyzer analyzer = new LogAnalyzer(logger);
    analyzer.Analyze("a.txt");
    logger.Received().Log("Filename too short: a.txt");
}

//using MOQ
[Test]
public void Analyze_TooShortFileName_CallLoggerMoq()
{
    var mockLogger = new Mock<ILogger>();
    LogAnalyzer analyzer = new LogAnalyzer(mockLogger.Object);
    analyzer.Analyze("a.txt");

    mockLogger.Verify(logger => logger.Log(It.Is<string>(error =>
        error.Contains("too short"))), Times.Once());
}
```

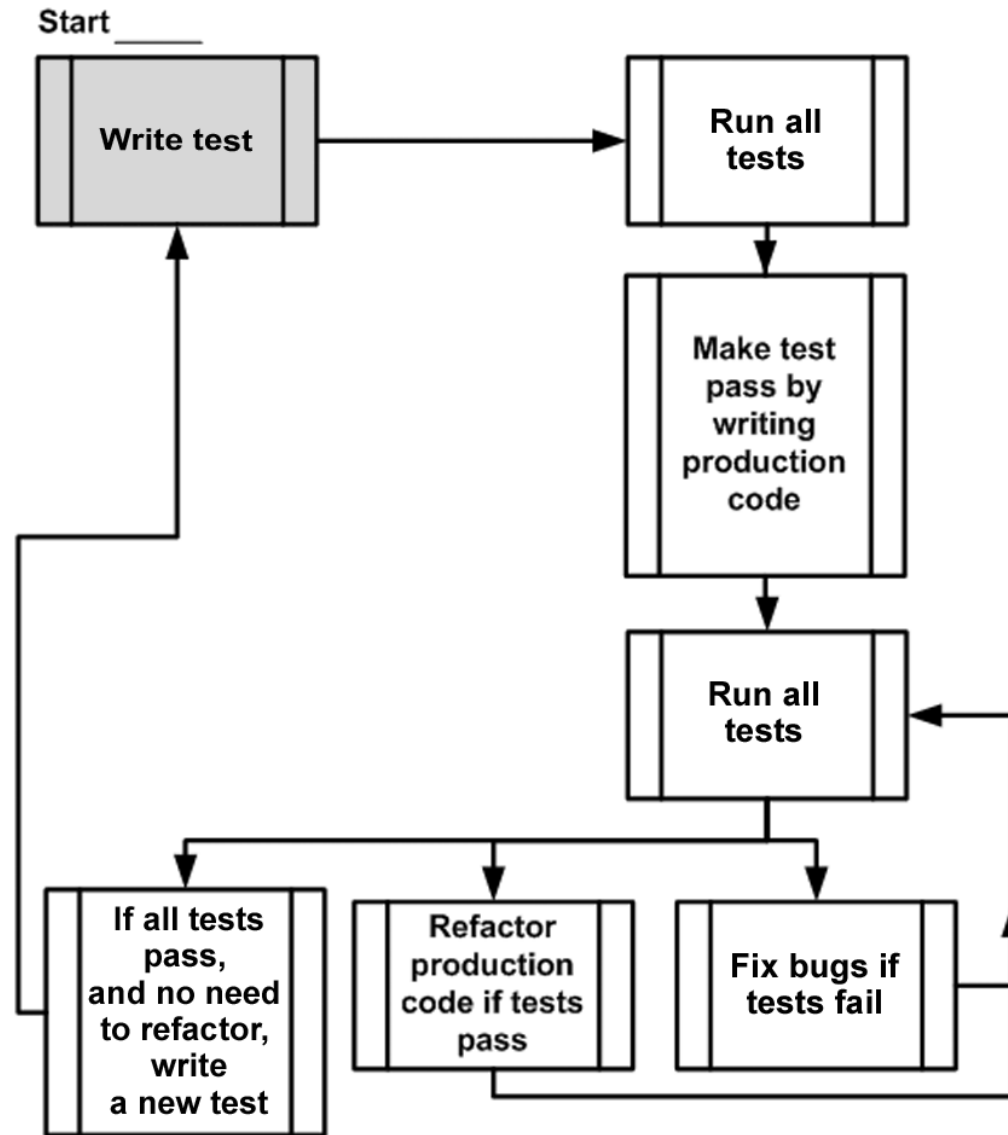
Changing or removing tests

- Codebase has a bug
 - Tests are fine. The safety net worked 😊
- Test has a bug
 - Fix as you see fit, then
 - Make sure it fails when it should: Comment a line in tested code / change line order now it should fail
 - Make sure it passes when it should: Change it back, now it should pass
- New / conflicting / invalid requirements
- API changes
- Duplicate tests

Legacy code

- It often refers to code that's hard to work with, hard to test, and usually even hard to read.
- “source code that relates to a no-longer supported or manufactured operating system or other computer technology,”
- A client defined legacy code as “code that works.”
- A popular definition is “code that has no tests.”

TDD



Final thoughts

- Test only one concern
- Separate unit and integration tests
- Compare objects instead of properties (generate the Equals() method)
- Tests are a form of documentation/specification for your code so make sure they are easy to understand
- A test should always run in its own little world, isolated from even the knowledge that other tests are out there, so none of the following:
 - Constrained test order: Tests expecting to be run in a specific order or expecting information from other test results
 - Hidden test call: Tests calling other tests
 - Shared-state corruption: Tests sharing in-memory state without rolling back
 - External shared-state corruption: Integration tests with shared resources and no rollback